

# claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

## Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf\_a2b97cff-dcd\agent-adf12bc6c307e0f25.jsonl`
- SHA-256: `4e15b8dad7c841b4ee96552d5b91cf35c22ffda2acc53cae37f804cfb83cb8d9`
- Source modified: `2026-06-21T13:36:52+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf\_a2b97cff-dcd`
- session\_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`

## Transcript

### 1. user (2026-06-21T13:34:48.456Z)

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector.  
Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI).  
Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

- pyproject.toml: hatchling build, namespace package (no backend/\_\_init\_\_.py), packages=["backend","training"], console script: indexer = backend.main:main. requires-python >=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo\_hybrid|wasb\_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb\_badminton\_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute\_target on AppConfig; backend/config/presets.py (PROFILE\_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/\* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/\* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi). The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.
- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP\_APP\_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).

khelsutra/docs/LOCAL\_APPLIANCE\_ARCHITECTURE.md is the live appliance design.  
Always ground claims in file:line or a doc path. Read the actual files; do not assume.

AREA: Storage + Compute backend registries (the local-OR-cloud axis). READ: backend/storage/ (\_\_init\_\_, ref.py, local, gcs, s3, gdrive) and backend/compute/ (registry, cloudrun). Answer: how input/output backends are selected; what creds each cloud backend needs and how they're supplied; how CloudRunCompute dispatches a job + polls completion; what a packaged app needs so a user can point at local files OR a gs://|s3://|gdrive source from a config/UI; which cloud SDKs are lazy-imported (so the base install stays light).

## 2. assistant (2026-06-21T13:34:51.307Z)

I'll investigate the storage and compute backend registries. Let me start by pulling the latest and reading the relevant files.

## 3. user (2026-06-21T13:34:54.587Z)

fatal: Cannot fast-forward to multiple branches.

## 4. user (2026-06-21T13:34:54.767Z)

```
backend/compute/:
total 32
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:20 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:04 ..
-rw-r--r-- 1 avidu 197609 2783 Jun 21 16:07 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:47 __pycache__
-rw-r--r-- 1 avidu 197609 2483 Jun 21 16:20 base.py
-rw-r--r-- 1 avidu 197609 8275 Jun 21 16:20 cloud_run.py
```

```
backend/storage/:
total 52
drwxr-xr-x 1 avidu 197609    0 Jun 21 13:37 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:04 ..
-rw-r--r-- 1 avidu 197609 6326 Jun 21 13:37 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:47 __pycache__
-rw-r--r-- 1 avidu 197609 2231 Jun 20 15:34 base.py
-rw-r--r-- 1 avidu 197609 2661 Jun 20 13:48 gcs.py
-rw-r--r-- 1 avidu 197609 6910 Jun 20 15:47 gdrive.py
-rw-r--r-- 1 avidu 197609 2756 Jun 20 13:48 local.py
-rw-r--r-- 1 avidu 197609 4455 Jun 21 13:23 ref.py
-rw-r--r-- 1 avidu 197609 3728 Jun 20 13:48 s3.py
```

## 5. assistant (2026-06-21T13:34:56.919Z)

Let me read all the storage and compute files in parallel.

## 6. user (2026-06-21T13:34:57.902Z)

```
1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13     from __future__ import annotations
14
```

```

15     import os
16     import tempfile
17     from typing import Any, Dict, Iterator, Optional
18
19     from backend.storage.base import StorageBackend
20     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22     __all__ = [
23         "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24         "get_storage", "fetch", "put", "list_uris",
25         "acquire_local_input", "input_is_local", "resolve_input_ref",
26     ]
27
28
29     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31         if storage_config is None:
32             return {}
33         if hasattr(storage_config, "model_dump"):
34             return storage_config.model_dump() # type: ignore[no-any-return]
35         return dict(storage_config)
36
37
38     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
40         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
41         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44         ``local://?`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45         sc = _storage_dict(storage_config)
46         uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                               input_root=sc.get("input_root", ""))
48         return parse_uri(uri)
49
50
51     def input_is_local(raw: str, storage_config: Any = None) -> bool:
52         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55         ``resolve_input_ref``)."""
56         return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59     def acquire_local_input(raw: str, storage_config: Any = None, *,
60                             scratch_parent: "str | None" = None) -> str:
61         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
65         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull

```

```

pattern the
68     worker's ``cmd_infer`` already ships."""
69     sc = _storage_dict(storage_config)
70     ref = resolve_input_ref(raw, sc)
71     if ref.backend == "local":
72         return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73     scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74     return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage
91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->

```

```

Iterator[str]:
123     """Yield the URI of every object under ``prefix_uri``."""
124     ref = parse_uri(prefix_uri)
125     for r in get_storage(ref.backend, config=config).list(ref):
126         yield r.uri
127

```

## 7. user (2026-06-21T13:34:58.366Z)

```

1     """The ``StorageBackend`` ABC ? one interface for local + object-store + Drive.
2
3     Mirrors the codebase's ABC+Registry idiom (cf. ``SegmenterRegistry``). The bucket
backends
4     live in sibling modules and lazy-import their cloud SDK so ``import backend.storage``
never
5     pulls ``boto3`` / ``google-cloud-storage`` (``gdrive`` is pure local-FS over a mounted
Drive ?
6     no SDK). See ``PLATFORM_ARCHITECTURE.md`` §3.1 and ``docs/DECOUPLED_COMPUTE/``.
7     """
8     from __future__ import annotations
9
10    import os
11    from abc import ABC, abstractmethod
12    from typing import Iterator
13
14    from backend.storage.ref import StorageRef, StorageStat
15
16
17    class StorageBackend(ABC):
18        """Backend-agnostic object storage. ``fetch_to_local`` + ``put`` are the two the
19        worker dispatcher needs; the rest support listing/checkpointing/serving."""
20
21        @abstractmethod
22        def fetch_to_local(self, ref: StorageRef, dst: "str | os.PathLike[str] | None" =
None,
23                        *, overwrite: bool = False) -> str:
24            """Make the object available as a local file; return its local path. For the
25            ``local`` backend this is an identity passthrough (the object is already
local)."""
26
27        @abstractmethod
28        def put(self, local_path: "str | os.PathLike[str]", ref: StorageRef, *,
29               content_type: "str | None" = None) -> StorageRef:
30            """Upload/copy a local file to ``ref``; return ``ref``."""
31
32        @abstractmethod
33        def exists(self, ref: StorageRef) -> bool: ...
34
35        @abstractmethod
36        def stat(self, ref: StorageRef) -> StorageStat: ...
37
38        @abstractmethod
39        def list(self, prefix: StorageRef) -> Iterator[StorageRef]:
40            """Yield a ``StorageRef`` per object under ``prefix``."""
41
42        @abstractmethod
43        def delete(self, ref: StorageRef) -> None: ...
44
45        def signed_url(self, ref: StorageRef, *, ttl: int = 3600, method: str = "GET") ->
str:
46            """A time-limited URL handing the bytes to a browser/LLM. Not every backend can
47            sign (``local`` cannot; Drive only emulates) ? callers must treat
48            ``NotImplementedError`` as expected and fall back to ``fetch_to_local``."""
49            raise NotImplementedError(f"{type(self).__name__} cannot produce signed URLs")
50

```

## 8. user (2026-06-21T13:34:59.316Z)

```
1      """Storage value objects + URI parsing for the pluggable storage layer.
2
3      A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5          <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6          local://<path>             -> local    (local:///abs or local://rel/p)
7          s3://<bucket>/<key>        -> s3       (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8          gcs://<bucket>/<key>      -> gcs       (gs:// accepted, normalized to gcs://)
9          gdrive://<path>           -> gdrive    (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53          """
54          s = str(uri)
55          m = _SCHEME_RE.match(s)
56          if not m:
```

```

57         return StorageRef("local", "", s, f"local://{s}")
58     scheme = m.group(1).lower()
59     rest = s[m.end():]
60     if scheme == "gs":
61         scheme = "gcs"
62     if scheme in ("local", "gdrive"):
63         # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64         return StorageRef(scheme, "", rest, s)
65     if scheme in _KNOWN:
66         bucket, _, key = rest.partition("/")
67         return StorageRef(scheme, bucket, key, s)
68     raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})")
69
70
71     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72         """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74         Precedence (first match wins):
75         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
(``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
78         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
80         ``"gcs://bucket/x"``).
81         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
passthrough for every
84         path used today, preserving byte-for-byte behaviour.
85         """
86         s = str(raw)
87         if _SCHEME_RE.match(s):          # 1. explicit scheme wins
88             return s
89         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90         #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91         #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\ ", or a Windows
drive prefix "X:".
92         if s.startswith((" /", "\\ ")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93             return s
94         if input_root:                    # 3. join a relative name under the configured
root
95             return input_root.rstrip("/") + "/" + s.lstrip("/")
96         if input_backend and input_backend != "local":    # 4. prefix a bucket/key with the
declared backend
97             return f"{input_backend}://{s}"
98         return s                          # 5. bare/relative local path ? unchanged
99

```

## 9. user (2026-06-21T13:34:59.785Z)

```

1     """Local-filesystem storage backend ? the default, zero new dependencies.
2
3     Wraps today's behaviour byte-for-byte: ``fetch_to_local`` is an identity passthrough
(the
4     file is already local), ``put`` is a copy (a no-op when src == dst). With
``backend='local'``
5     the whole pipeline behaves exactly as before the storage layer existed.

```

```

6      """
7      from __future__ import annotations
8
9      import os
10     import shutil
11     from datetime import datetime
12     from typing import Iterator
13
14     from backend.storage.base import StorageBackend
15     from backend.storage.ref import StorageRef, StorageStat
16
17
18     class LocalStorage(StorageBackend):
19         def __init__(self, *, base_dir: str = "", **_ignored) -> None:
20             # Relative keys resolve under base_dir when set; absolute keys are used as-is.
21             self.base_dir = base_dir
22
23         def _path(self, ref: StorageRef) -> str:
24             p = ref.key
25             if self.base_dir and not os.path.isabs(p):
26                 return os.path.join(self.base_dir, p)
27             return p
28
29         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
30             # The object is already on the local FS ? return its path (identity). ``dst`` is
31             # honoured only if it names a *different* location (a copy is then made).
32             src = self._path(ref)
33             if dst is None or os.path.abspath(str(dst)) == os.path.abspath(src):
34                 return src
35             dst = str(dst)
36             if os.path.exists(dst) and not overwrite:
37                 return dst
38             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
39             shutil.copyfile(src, dst)
40             return dst
41
42         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
43             dst = self._path(ref)
44             if os.path.abspath(str(local_path)) != os.path.abspath(dst):
45                 os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46                 shutil.copyfile(str(local_path), dst)
47             return ref
48
49         def exists(self, ref) -> bool:
50             return os.path.exists(self._path(ref))
51
52         def stat(self, ref) -> StorageStat:
53             st = os.stat(self._path(ref))
54             return StorageStat(size=st.st_size,
55 modified=datetime.fromtimestamp(st.st_mtime))
56
57         def list(self, prefix) -> Iterator[StorageRef]:
58             base = self._path(prefix)
59             if os.path.isdir(base):
60                 for root, _dirs, files in os.walk(base):
61                     for fn in sorted(files):
62                         full = os.path.join(root, fn)
63                         yield StorageRef("local", "", full, f"local://{full}")
64             elif os.path.exists(base):
65                 yield StorageRef("local", "", base, f"local://{base}")
66
67         def delete(self, ref) -> None:
68             path = self._path(ref)
69             if os.path.exists(path):
70                 os.remove(path)

```



**10. user (2026-06-21T13:35:00.257Z)**

```

1      """Google Cloud Storage backend ? first-class object buckets.
2
3      Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4      install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5      (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6      never from config. A ``client`` may be injected for tests.
7      """
8      from __future__ import annotations
9
10     from datetime import timedelta
11     from typing import Any, Iterator, Optional
12
13     from backend.storage.base import StorageBackend
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class GCSStorage(StorageBackend):
18         def __init__(self, *, client: Any = None, project: Optional[str] = None,
19             signed_url_ttl: int = 3600, **_ignored) -> None:
20             self._ttl = signed_url_ttl
21             if client is not None:
22                 self._client = client
23             return
24             from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25                 self._client = storage.Client(project=project)
26
27         def _blob(self, ref: StorageRef):
28             return self._client.bucket(ref.bucket).blob(ref.key)
29
30         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31             if dst is None:
32                 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33             import os
34             dst = str(dst)
35             if os.path.exists(dst) and not overwrite:
36                 return dst
37             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38             self._blob(ref).download_to_filename(dst)
39             return dst
40
41         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42             self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43             return ref
44
45         def exists(self, ref) -> bool:
46             return bool(self._blob(ref).exists())
47
48         def stat(self, ref) -> StorageStat:
49             b = self._blob(ref)
50             b.reload()
51             return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                 content_type=b.content_type)
53
54         def list(self, prefix) -> Iterator[StorageRef]:
55             for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56                 yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")

```

```

57
58     def delete(self, ref) -> None:
59         self._blob(ref).delete()
60
61     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62         return self._blob(ref).generate_signed_url(
63             version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64

```

## 11. user (2026-06-21T13:35:00.722Z)

```

1     """S3 (and S3-compatible) storage backend ? first-class object buckets.
2
3     One backend covers **AWS S3, Cloudflare R2, DigitalOcean Spaces, and MinIO** ? they
differ
4     only by ``endpoint_url`` (+ region / addressing style). Uses ``boto3``, imported lazily
so
5     ``import backend.storage`` never pulls it; install with the ``storage-s3`` extra.
6
7     Credentials are NEVER read from config ? only ``endpoint_url`` / ``region`` / addressing
8     style. boto3's default chain supplies creds (``AWS_ACCESS_KEY_ID`` /
``AWS_SECRET_ACCESS_KEY``
9     / ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client`` may be injected
10    (tests pass a fake; production lets the constructor build one via boto3).
11    """
12    from __future__ import annotations
13
14    import os
15    from typing import Any, Iterator, Optional
16
17    from backend.storage.base import StorageBackend
18    from backend.storage.ref import StorageRef, StorageStat
19
20
21    class S3Storage(StorageBackend):
22        def __init__(self, *, client: Any = None, endpoint_url: Optional[str] = None,
23                     region: Optional[str] = None, addressing_style: str = "auto",
24                     signed_url_ttl: int = 3600, **_ignored) -> None:
25            self._ttl = signed_url_ttl
26            if client is not None:
27                self._client = client
28            return
29
30        import boto3 # type: ignore # lazy: only when an s3:// URI is actually used
31        from botocore.config import Config # type: ignore
32        self._client = boto3.client(
33            "s3",
34            endpoint_url=endpoint_url or os.environ.get("AWS_ENDPOINT_URL_S3") or None,
35            region_name=region,
36            config=Config(s3={"addressing_style": addressing_style}),
37        )
38
39        def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
40            if dst is None:
41                raise ValueError("S3Storage.fetch_to_local requires a dst path")
42            dst = str(dst)
43            if os.path.exists(dst) and not overwrite:
44                return dst
45            os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
46            self._client.download_file(ref.bucket, ref.key, dst)
47            return dst
48
49        def put(self, local_path, ref, *, content_type=None) -> StorageRef:
50            extra = {"ContentType": content_type} if content_type else None
51            self._client.upload_file(str(local_path), ref.bucket, ref.key, ExtraArgs=extra)
52            return ref

```

```

52
53     def exists(self, ref) -> bool:
54         try:
55             self._client.head_object(Bucket=ref.bucket, Key=ref.key)
56             return True
57         except Exception: # noqa: BLE001 - any boto/client error means "not found /
unreachable"
58             return False
59
60     def stat(self, ref) -> StorageStat:
61         h = self._client.head_object(Bucket=ref.bucket, Key=ref.key)
62         return StorageStat(size=int(h.get("ContentLength", 0)),
modified=h.get("LastModified"),
63                             etag=h.get("ETag"), content_type=h.get("ContentType"))
64
65     def list(self, prefix) -> Iterator[StorageRef]:
66         paginator = self._client.get_paginator("list_objects_v2")
67         for page in paginator.paginate(Bucket=prefix.bucket, Prefix=prefix.key):
68             for obj in page.get("Contents", []) or []:
69                 key = obj["Key"]
70                 yield StorageRef("s3", prefix.bucket, key,
f"s3://{prefix.bucket}/{key}")
71
72     def delete(self, ref) -> None:
73         self._client.delete_object(Bucket=ref.bucket, Key=ref.key)
74
75     def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
76         op = "get_object" if method.upper() == "GET" else "put_object"
77         return self._client.generate_presigned_url(
78             op, Params={"Bucket": ref.bucket, "Key": ref.key}, ExpiresIn=ttl or
self._ttl)
79

```

## 12. user (2026-06-21T13:35:01.679Z)

```

1     """Google Drive backend ? reads Drive files through a **locally mounted** Google Drive.
2
3     We make a deliberate, baked-in assumption: **to use Google Drive files you mount Google
Drive
4     for Desktop**, which exposes Drive as an ordinary filesystem path. That's a fair
requirement ? it
5     needs no service account, no API quota, no OAuth, and Drive streams + caches bytes on
access. So
6     ``gdrive://`` is just a *portable, OS-agnostic alias* over that mount: it names a path
**relative to
7     your "My Drive" folder**, and everything else is plain local-filesystem I/O (the same
code the
8     ``local`` backend uses).
9
10         gdrive://Sharing/clip.mp4    ->    <mount>/My Drive/Sharing/clip.mp4
11
12     Why an alias instead of just a raw path? Portability: the mount root differs per machine
13     (Windows ``G:\My Drive``, macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My
Drive``), so the same
14     ``gdrive://`` URI resolves on any box with Drive mounted. The root is **auto-detected**,
or set it
15     explicitly via ``GDRIVE_MOUNT_ROOT`` / ``storage.gdrive.mount_root`` (a path, not a
secret).
16
17     A headless box (a GPU VM, CI) has **no** mount ? there, stage the file to a bucket and
use
18     ``gcs://`` / ``s3://``, or ``gdown <file-id>`` to a local path. This backend
intentionally does NOT
19     talk to the Drive API. When the mount (or a file in it) is missing, errors point the
user at the

```

```

20     fix: install Drive for Desktop and include the folder in syncing so it can be pulled
locally.
21     """
22     from __future__ import annotations
23
24     import glob
25     import os
26     import string
27     from typing import Iterator, Optional
28
29     from backend.storage.base import StorageBackend
30     from backend.storage.local import LocalStorage
31     from backend.storage.ref import StorageRef, StorageStat
32
33     _INSTALL_URL = "https://www.google.com/drive/download/"
34
35     # Shared remedy line ? how to make a folder available locally. Drive for Desktop streams
files,
36     # but a folder only appears once it's part of your synced Drive (and a *shared* folder
must first
37     # be added to "My Drive"). The same fix resolves both "not mounted" and "mounted but
missing".
38     _SYNC_REMEDY = (
39         "make sure the folder is INCLUDED in syncing so its files can be pulled locally: in
Google "
40         "Drive for Desktop keep 'Stream files' on, and for a folder shared with you, open
Drive in the "
41         "browser, right-click it -> 'Add shortcut to Drive' so it appears under 'My Drive'.
Drive then "
42         "downloads the bytes on first access."
43     )
44
45
46     def _not_mounted_msg(tried: Optional[str]) -> str:
47         return (
48             "Google Drive is not mounted, so the gdrive:// backend cannot resolve any
path.\n"
49             f"  Remedy:\n"
50             f"  1. Install Google Drive for Desktop and sign in: {_INSTALL_URL}\n"
51             f"  2. Then {_SYNC_REMEDY}\n"
52             f"  3. If your 'My Drive' is not at the default location (Windows G:\\My Drive,
macOS "
53             f"~/Library/CloudStorage/GoogleDrive-<acct>/My Drive), set GDRIVE_MOUNT_ROOT or "
54             f"storage.gdrive.mount_root to point at it (auto-detect tried: {tried!r})."
55         )
56
57
58     def _not_synced_msg(root: str, rel: str, uri: str) -> str:
59         return (
60             f"Google Drive is mounted ({root!r}) but {uri!r} is not present locally "
61             f"(expected at {os.path.join(root, *rel.split('/')!r}).\n"
62             f"  This usually means the folder is not synced yet ? {_SYNC_REMEDY}\n"
63             f"  (Install/repair Drive for Desktop if needed: {_INSTALL_URL})"
64         )
65
66
67     def _autodetect_mount_root() -> Optional[str]:
68         """Find a mounted Google Drive 'My Drive' folder, or ``None``. Covers the Google
Drive for
69         Desktop defaults: macOS ``~/Library/CloudStorage/GoogleDrive-<acct>/My Drive`` and
Windows
70         ``<letter>:\\My Drive`` (``G:`` is the install default). Linux/other ? configure
explicitly."""
71         home = os.path.expanduser("~")
72         for p in sorted(glob.glob(os.path.join(home, "Library", "CloudStorage",

```

```

"GoogleDrive*", "My Drive"))):
73         if os.path.isdir(p):
74             return p
75     if os.name == "nt":
76         # G is the Drive-for-Desktop default; then scan the rest of the assignable
letters.
77         for letter in ["G"] + [c for c in string.ascii_uppercase if c != "G"]:
78             p = f"{letter}:\\My Drive"
79             if os.path.isdir(p):
80                 return p
81     return None
82
83
84     class GDriveStorage(StorageBackend):
85         def __init__(self, *, mount_root: Optional[str] = None, **_ignored) -> None:
86             root = mount_root or os.environ.get("GDRIVE_MOUNT_ROOT") or
_autodetect_mount_root()
87             if not root or not os.path.isdir(root):
88                 raise RuntimeError(_not_mounted_msg(root))
89             self._root = os.path.abspath(root)
90             self._local = LocalStorage(base_dir=self._root) # all I/O is plain local-FS
under the mount
91
92         def _local_ref(self, ref: StorageRef) -> StorageRef:
93             """Translate a ``gdrive://<rel>`` ref into a ``local`` ref keyed by an OS path
relative to
94             the mount root (``LocalStorage`` joins it under ``base_dir``)."""
95             rel = ref.key.lstrip("/")
96             key = os.path.join(*rel.split("/")) if rel else ""
97             return StorageRef("local", "", key, f"local://{key}")
98
99         def _require_present(self, ref: StorageRef) -> None:
100             """Guard the read paths: a path missing from the mount almost always means the
folder
101             isn't synced ? raise with that remedy instead of a cryptic FileNotFoundError
downstream."""
102             rel = ref.key.lstrip("/")
103             abs_path = os.path.join(self._root, *rel.split("/")) if rel else self._root
104             if not os.path.exists(abs_path):
105                 raise FileNotFoundError(_not_synced_msg(self._root, rel, ref.uri))
106
107         def _to_gdrive_uri(self, abs_path: str) -> str:
108             """The inverse: an absolute path under the mount ? its ``gdrive://`` URI."""
109             rel = os.path.relpath(abs_path, self._root).replace(os.sep, "/")
110             return f"gdrive://{rel}"
111
112         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
113             # The file is already local (via the mount): no dst => identity passthrough to
the mount
114             # path; a dst => copy out (a byte-stable scratch copy), exactly as the local
backend does.
115             self._require_present(ref)
116             return self._local.fetch_to_local(self._local_ref(ref), dst,
overwrite=overwrite)
117
118         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
119             self._local.put(local_path, self._local_ref(ref), content_type=content_type)
120             return ref
121
122         def exists(self, ref) -> bool:
123             return self._local.exists(self._local_ref(ref))
124
125         def stat(self, ref) -> StorageStat:
126             self._require_present(ref)
127             return self._local.stat(self._local_ref(ref))

```

```

128
129     def list(self, prefix) -> Iterator[StorageRef]:
130         for r in self._local.list(self._local_ref(prefix)):
131             uri = self._to_gdrive_uri(r.key)
132             yield StorageRef("gdrive", "", uri[len("gdrive://"):], uri)
133
134     def delete(self, ref) -> None:
135         self._local.delete(self._local_ref(ref))
136

```

### 13. user (2026-06-21T13:35:02.132Z)

```

1     """Pluggable compute layer: in-process (default) + a Cloud Run GPU Job dispatch shim.
2
3     COMPUTE_DECOUPLED_SERVING M6. ``get_compute_backend(config)`` is the seam: it returns a
4     ``ComputeBackend`` ONLY when a non-local ``deployment.compute_target`` is selected, and
5     ``None``
6     otherwise ? ``None`` means "run the worker's existing in-process path" (today's
7     behaviour, byte
8     identical). So the seam is **default-OFF**: with no profile / ``local_gpu`` /
9     ``cpu_mock`` nothing
10    changes. Mirrors ``backend/storage``'s lazy factory idiom (cloud SDKs imported only when
11    needed).
12    """
13
14    from __future__ import annotations
15
16    import os
17    from typing import Any, Optional
18
19    from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
20    from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy
21
22    __all__ = [
23        "ComputeBackend", "ComputeJobSpec", "ComputeResult", "get_compute_backend",
24    ]
25
26    def _storage_dict(config: Any) -> dict:
27        storage = getattr(config, "storage", None)
28        if storage is None:
29            return {}
30        if hasattr(storage, "model_dump"):
31            return storage.model_dump() # type: ignore[no-any-return]
32        return dict(storage) if isinstance(storage, dict) else {}
33
34    def get_compute_backend(
35        config: Any,
36        *,
37        run_client: Optional[Any] = None,
38        policy: ExecutionPolicy = DEFAULT_POLICY,
39        token: Optional[str] = None,
40    ) -> Optional[ComputeBackend]:
41        """Return the compute backend for ``config.deployment.compute_target``, or ``None``
42        to run
43        in-process (the default-OFF path).
44
45        Only ``compute_target == "cloud_run"`` returns a non-None backend (a
46        ``CloudRunCompute``). Its
47        Cloud Run coordinates come from the control-plane env (``CLOUD_RUN_JOB`` /
48        ``CLOUD_RUN_REGION``
49        / ``GOOGLE_CLOUD_PROJECT``) so no secrets live in config. ``run_client`` is
50        injectable (tests
51        pass a fake; production defaults to the real ``google-cloud-run`` client, owner-

```

```

verified on M7).
46     """
47     target = getattr(getattr(config, "deployment", None), "compute_target", "") or ""
48     if target != "cloud_run":
49         return None # local_gpu / cpu_mock / "" ? run inline (today's path, unchanged)
50
51     # Lazy import so the cloud SDK shim is only pulled when a cloud target is actually
52     selected.
53     from backend.compute.cloud_run import CloudRunCompute, GoogleCloudRunJobClient
54
55     job_name = os.environ.get("CLOUD_RUN_JOB", "rally-worker")
56     region = os.environ.get("CLOUD_RUN_REGION", "asia-south1")
57     project = os.environ.get("GOOGLE_CLOUD_PROJECT")
58     client = run_client if run_client is not None else GoogleCloudRunJobClient()
59     return CloudRunCompute(
60         run_client=client, job_name=job_name, region=region, project=project,
61         storage_config=storage_dict(config), policy=policy, token=token,
62     )

```

#### 14. user (2026-06-21T13:35:02.646Z)

```

1     """The ``ComputeBackend`` ABC ? one interface for *where* the rally core runs.
2
3     COMPUTE_DECOUPLED_SERVING M6. Mirrors the ``StorageBackend`` ABC+factory idiom
4     (``backend/storage/``): a ``ComputeBackend`` decides **where** the pure
5     DETECT?WINDOW?STITCH
6     core executes ? in-process on the local box (truly-local) or on a **Cloud Run GPU Job**
7     (cloud-serving) ? without the core ever branching on the profile (D1).
8
9     The seam is **default-OFF**: only ``deployment.compute_target == "cloud_run"`` selects a
10    non-local backend. Every other target (``"" / ``local_gpu`` / ``cpu_mock``) keeps the
11    worker's
12    existing in-process path **byte-identical** ? the factory returns ``None`` (= "run
13    inline").
14    """
15
16    from __future__ import annotations
17
18    from abc import ABC, abstractmethod
19    from dataclasses import dataclass, field
20    from typing import Optional
21
22    @dataclass(frozen=True)
23    class ComputeJobSpec:
24        """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
25        (re)compute
26        from the SAME input bytes, so the output always lands at
27        ``<out_uri>/outputs/<video_id>/`` ?
28        deterministic regardless of where the job ran."""
29
30        video_uri: str # input (gs://? for cloud; a path/local://
31        for local)
32        out_uri: str # output ROOT (gs://? bucket for cloud)
33        segmenter: Optional[str] = None
34        # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays
35        frozen/hashable.
36        config_overrides: tuple[str, ...] = ()
37
38    @dataclass(frozen=True)
39    class ComputeResult:
40        """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit
41        code

```

```

36         (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch
timeout or an
37         empty/unreadable completion marker** (the cloud path only). ``video_id`` is known
once the job's
38         done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed.""
39
40         returncode: int
41         outputs_uri: str = ""
42         video_id: str = ""
43         report: dict = field(default_factory=dict)
44
45
46     class ComputeBackend(ABC):
47         """Backend-agnostic compute. ``run_infer`` dispatches ONE inference job and returns
only once
48         its outputs are durably available (bucket-poll for cloud), so a caller treats local
and cloud
49         execution identically."""
50
51         @abstractmethod
52         def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
53             """Run ``spec`` to completion and return its ``ComputeResult``."""
54

```

## 15. user (2026-06-21T13:35:03.007Z)

```

1         """``CloudRunCompute`` ? dispatch one inference job to a Cloud Run GPU **Job** and
bucket-poll
2         for its completion (COMPUTE_DECOUPLED_SERVING M6, decisions D16 + D7).
3
4         Flow (the cloud-serving compute path):
5         1. **Dispatch** a Cloud Run Job execution whose container runs ``python -m
backend.worker infer
6             --video-uri <gs> --out-uri <gs> --done-uri <gs> ?`` (the SAME worker, on the L4
GPU).
7         2. **Bucket-poll** ``done_uri`` via ``execution_policy.poll_until`` + the storage
layer (D16 ? the
8             completion signal is a tiny marker object the worker writes, NOT the Cloud Run API
status, so
9             it is storage-backed and reuses the existing park/resume primitives). Scale-to-zero
(D7): the
10            cold-start is absorbed by the async poll.
11         3. **Read** the marker (it carries ``video_id`` + the worker's returncode) ? fetch
12            ``outputs/<video_id>/report.json`` ? ``ComputeResult``.
13
14         The dispatched container is forced to run **INLINE** (``compute_target=local_gpu`` +
native detector)
15         so it never recursively re-dispatches ? the dispatcher decides cloud, the container just
computes.
16
17         The Cloud Run trigger is an **injected client** (``CloudRunJobClient``) so the whole
shim is unit-tested
18         with a fake (no GCP). The real ``GoogleCloudRunJobClient`` lazy-imports ``google-cloud-
run`` and is
19         owner-verified on the M7 real run.
20         """
21
22         from __future__ import annotations
23
24         import json
25         import logging
26         import os
27         import tempfile
28         import uuid

```



```

29     from abc import ABC, abstractmethod
30     from typing import Any, List, Optional
31
32     from backend.compute.base import ComputeBackend, ComputeJobSpec, ComputeResult
33     from backend.infrastructure.execution_policy import DEFAULT_POLICY, ExecutionPolicy,
poll_until
34     from backend.storage import fetch, parse_uri
35     from backend.storage import get_storage
36
37     LOG = logging.getLogger("compute.cloud_run")
38
39     # The container worker must run INLINE on the GPU ? never re-enter the cloud-dispatch
branch.
40     # (compute_target=local_gpu disarms the cmd_infer dispatch guard; native = the L4 WASB
runner.)
41     _DEFAULT_CONTAINER_OVERRIDES: tuple[str, ...] = (
42         "deployment.compute_target=local_gpu",
43         "indexing.detector_impl=native",
44     )
45
46
47     class CloudRunJobClient(ABC):
48         """Triggers a Cloud Run **Job** execution with per-execution container arg
overrides.
49
50         Abstracted so ``CloudRunCompute`` is testable with a fake (no GCP). A real
implementation
51         overrides the job's container ``args`` for this execution and starts it."""
52
53         @abstractmethod
54         def run_job(self, job_name: str, argv: List[str], *, region: str,
55                     project: Optional[str] = None) -> str:
56             """Start a Cloud Run Job execution running ``python -m backend.worker <argv>``;
return an
57             execution id/name (for logs). Does NOT block on completion (the caller bucket-
polls)."""
58
59
60     class GoogleCloudRunJobClient(CloudRunJobClient):
61         """Real client ? lazy-imports ``google-cloud-run``. Owner-verified on the M7 real
run; CI uses
62         a fake, so this code path is never exercised in tests (the SDK isn't a test
dependency)."""
63
64         def run_job(self, job_name: str, argv: List[str], *, region: str,
65                     project: Optional[str] = None) -> str:
66             try:
67                 from google.cloud import run_v2 # type: ignore[attr-defined]
68             except Exception as exc: # pragma: no cover - real SDK not present in CI
69                 raise RuntimeError(
70                     "google-cloud-run is required for CloudRunCompute's real client; install
it on the "
71                     "control plane (it is intentionally NOT a CI/test dependency).")
72             ) from exc
73             client = run_v2.JobsClient() # pragma: no cover - exercised only on the real M7
run
74             name = client.job_path(project, region, job_name)
75             overrides = run_v2.RunJobRequest.Overrides(
76                 container_overrides=[run_v2.RunJobRequest.Overrides.ContainerOverride(args=argv)]
77             )
78             op = client.run_job(request=run_v2.RunJobRequest(name=name,
overrides=overrides))
79             return getattr(op, "metadata", None) and getattr(op.metadata, "name", "") or
"execution"

```

```

80
81
82 class CloudRunCompute(ComputeBackend):
83     """Dispatch to a Cloud Run GPU Job + bucket-poll for completion."""
84
85     def __init__(self, *, run_client: CloudRunJobClient, job_name: str, region: str,
86                  project: Optional[str] = None, storage_config: Optional[dict] = None,
87                  policy: ExecutionPolicy = DEFAULT_POLICY,
88                  token: Optional[str] = None,
89                  container_overrides: Optional[tuple[str, ...]] = None) -> None:
90         self._client = run_client
91         self._job_name = job_name
92         self._region = region
93         self._project = project
94         self._storage_config = storage_config or {}
95         self._policy = policy
96         # The dispatch token keys the done-marker so concurrent jobs to one bucket don't
collide.
97         # None ? a fresh uuid4 per run_infer call (prod); injected ? deterministic
(tests).
98         self._token = token
99         self._container_overrides = (
100             _DEFAULT_CONTAINER_OVERRIDES if container_overrides is None else
tuple(container_overrides)
101         )
102
103     def run_infer(self, spec: ComputeJobSpec) -> ComputeResult:
104         out_root = spec.out_uri.rstrip("/")
105         token = self._token or uuid.uuid4().hex
106         done_uri = f"{out_root}/_dispatch/{token}.done"
107         argv: List[str] = ["infer", "--video-uri", spec.video_uri, "--out-uri",
spec.out_uri,
108                             "--done-uri", done_uri]
109         if spec.segmenter:
110             argv += ["--segmenter", spec.segmenter]
111         for kv in (*spec.config_overrides, *self._container_overrides):
112             argv += ["--set", kv]
113
114         execution = self._client.run_job(self._job_name, argv, region=self._region,
115                                         project=self._project)
116         LOG.info("cloud-run: dispatched job=%s exec=%s; bucket-polling %s",
117                 self._job_name, execution, done_uri)
118
119         marker = poll_until(lambda: self._read_marker(done_uri), self._policy,
120                             label="cloud-run-infer", logger=LOG)
121         video_id = str(marker.get("video_id", ""))
122         # A MISSING returncode defaults to FAILURE (1), never success ? a present-but-
unreadable /
123         # garbled marker must not masquerade as rc=0. (The worker always writes both
fields.)
124         rc = int(marker.get("returncode", 1))
125         if not video_id:
126             LOG.warning("cloud-run: job=%s completion marker present but
empty/unreadable (no "
127                         "video_id) ? treating as failure", self._job_name)
128         rc = rc or 1
129         outputs_uri = f"{out_root}/outputs/{video_id}/" if video_id else ""
130         report = self._read_json(f"{outputs_uri}report.json") if outputs_uri else {}
131         LOG.info("cloud-run: job=%s done (video_id=%s rc=%d)", self._job_name, video_id,
rc)
132         return ComputeResult(returncode=rc, outputs_uri=outputs_uri, video_id=video_id,
report=report)
133
134         # --- storage-backed completion signal (bucket-poll) ---
135         def _read_marker(self, done_uri: str) -> Optional[dict]:

```

```

136         """The done-marker dict if the worker has written it, else None ? poll_until
keeps waiting."""
137         ref = parse_uri(done_uri)
138         if not get_storage(ref.backend, config=self._storage_config).exists(ref):
139             return None
140         return self._read_json(done_uri) or {} # empty dict still terminates the poll
(present == done)
141
142     def _read_json(self, uri: str) -> dict:
143         # Download to a real local dst FIRST: the cloud backends (gcs/s3) REQUIRE a dst
? a None dst
144         # raises ValueError ? unlike local's identity passthrough. Without an explicit
dst this read
145         # would silently return {} for every gs://|s3:// object (the only backends this
shim targets).
146         try:
147             ref = parse_uri(uri)
148             dst = os.path.join(tempfile.mkdtemp(prefix="cr-read-"), ref.name or
"obj.json")
149             local = fetch(uri, dst, config=self._storage_config)
150             with open(local, "r", encoding="utf-8") as f:
151                 data: Any = json.load(f)
152             return data if isinstance(data, dict) else {}
153         except Exception:
154             LOG.warning("cloud-run: could not read %s", uri, exc_info=True)
155             return {}
156

```

## 16. assistant (2026-06-21T13:35:06.645Z)

I have a clear picture of the storage and compute layers. Let me check how these are wired into config (StorageConfig, presets, deployment) and the worker/API boundary to understand backend selection and creds supply end-to-end.

## 17. user (2026-06-21T13:35:07.769Z)

```

total 76
drwxr-xr-x 1 avidu 197609    0 Jun 21 18:48 .
drwxr-xr-x 1 avidu 197609    0 Jun 21 19:04 ..
-rw-r--r-- 1 avidu 197609 1025 Jun 21 15:34 __init__.py
drwxr-xr-x 1 avidu 197609    0 Jun 21 16:47 __pycache__
-rw-r--r-- 1 avidu 197609  9762 Jun 21 13:00 loader.py
-rw-r--r-- 1 avidu 197609 26870 Jun 21 18:48 models.py
-rw-r--r-- 1 avidu 197609  6133 Jun 21 15:34 presets.py
-rw-r--r-- 1 avidu 197609  9005 Jun 21 15:53 startup.py

```

## 18. user (2026-06-21T13:35:08.588Z)

```

Found 16 files
backend\main.py
backend\config\models.py
docs\COMPUTE_DECOUPLED_SERVING\README.md
tests\test_compute_backend.py
deploy\cloudrun\README.md
backend\compute\base.py
backend\worker\__main__.py
backend\compute\cloud_run.py
backend\compute\__init__.py
backend\config\__init__.py
backend\config\presets.py
tests\test_input_source.py
backend\storage\__init__.py
backend\storage\ref.py
docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md

```

tests\test\_storage.py

## 19. user (2026-06-21T13:35:11.285Z)

```
1      """Deployment-profile presets + environment auto-detection (COMPUTE_DECOUPLED_SERVING
M1).
2
3      A *deployment profile* is a single switch that applies a coherent BUNDLE of existing
4      config knobs, so the truly-local / cloud-serving / cpu-mock modes don't drift apart one
5      knob at a time (the design in ``docs/COMPUTE_DECOUPLED_SERVING/README.md`` §4.3).
6
7      Precedence (lowest ? highest), realised by the loader::
8
9          built-in model defaults < profile preset < config.json < env < worker --set
10
11     This module is **pure data + pure functions** ? no IO, and (critically) **no torch
import at
12     module load**. The loader applies a preset ONLY when a profile is EXPLICITLY selected
13     (``config.json`` ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); with no
14     profile selected the loader is byte-identical to the pre-M1 behaviour (``profile=""`` ==
15     today). Auto-detect only ever **suggests** a profile ? it never auto-applies one (D15:
cloud
16     spend is never entered silently).
17     """
18
19     from __future__ import annotations
20
21     import copy
22     import os
23     from typing import Any, Mapping, Optional
24
25     # Canonical enum values, defined ONCE here. ``models.DeploymentConfig``'s ``Literal``
mirrors
26     # these and ``tests/test_config_deployment.py`` pins parity so the two can never drift.
27     PROFILES: tuple[str, ...] = ("truly-local", "cloud-serving", "cpu-mock")
28     COMPUTE_TARGETS: tuple[str, ...] = ("local_gpu", "cloud_run", "cpu_mock")
29
30     # Each profile maps 1:1 to its canonical compute_target. Used to apply the preset and to
warn
31     # when config.json explicitly overrides compute_target into a state inconsistent with
the profile.
32     COMPUTE_TARGET_FOR_PROFILE: dict[str, str] = {
33         "truly-local": "local_gpu",
34         "cloud-serving": "cloud_run",
35         "cpu-mock": "cpu_mock",
36     }
37
38     # A preset is a partial, nested override dict over the model defaults. It touches ONLY
knobs
39     # that EXIST today ? M1 is pure/additive: the new
``deployment.{profile,compute_target}`` plus
40     # ``indexing.{detector_impl,skip_ai_handoff}`` and ``storage.backend``. Later milestones
extend
41     # these bundles ? ``input_backend`` (M2) and the configurable output/workspace base (M3,
42     # Launch-Report-gated) ? deliberately NOT set here so M1 never pre-empts a gated flip.
43     PROFILE_PRESETS: dict[str, dict[str, Any]] = {
44         "truly-local": {
45             "deployment": {"profile": "truly-local", "compute_target": "local_gpu"},
46             # detector_impl stays "auto" (the runner picks WSL vs native by env on the GPU
box).
47             # Zero-cloud / privacy-first ? run fully local with no Gemini handoff.
48             "indexing": {"skip_ai_handoff": True},
49             "storage": {"backend": "local"},
50         },
51         "cloud-serving": {
```

```

52         "deployment": {"profile": "cloud-serving", "compute_target": "cloud_run"},
53         # Linux L4 (cuda); Gemini handoff ON until the owned model clears the #172 floor
(D11).
54         "indexing": {"detector_impl": "native", "skip_ai_handoff": False},
55         "storage": {"backend": "gcs"},
56     },
57     "cpu-mock": {
58         "deployment": {"profile": "cpu-mock", "compute_target": "cpu_mock"},
59         # Deterministic CPU mock ? no GPU/WSL, no provider/API. The CI / regression
profile.
60         "indexing": {"detector_impl": "stub", "skip_ai_handoff": True},
61         "storage": {"backend": "local"},
62     },
63 }
64
65
66 def is_known_profile(profile: str) -> bool:
67     """True for a non-empty, recognised profile name."""
68     return bool(profile) and profile in PROFILE_PRESETS
69
70
71 def preset_for(profile: str) -> dict[str, Any]:
72     """Return a deep COPY of the preset bundle for ``profile`` (so callers can't mutate
the
73     module-level table), or ``{}`` for ``""`` / an unknown profile."""
74     return copy.deepcopy(PROFILE_PRESETS.get(profile, {}))
75
76
77 def _truthy(val: Optional[str]) -> bool:
78     """Common env-var truthiness (``1/true/yes/on``, case/space-insensitive)."""
79     if val is None:
80         return False
81     return val.strip().lower() in {"1", "true", "yes", "on"}
82
83
84 def _cuda_available() -> bool:
85     """Best-effort CUDA probe. torch is heavy + optional, so it is imported lazily here
and
86     ONLY when a caller opts into ``probe_cuda``; any import/runtime failure means 'no
GPU'."""
87     try:
88         # Intentionally lazy + local: keeps the config-load path torch-free (torch is
heavy
89         # and optional). Only reached when a caller opts into probe_cuda.
90         import torch
91
92         return bool(torch.cuda.is_available())
93     except Exception:
94         return False
95
96
97 def probe_cuda_available() -> bool:
98     """Public best-effort CUDA probe (lazy torch import; ``False`` on any failure).
99
100     Exposed for the per-profile startup checks (``backend/config/startup.py``) so a
caller
101     that *wants* the GPU heads-up can pass the result in without the startup module
importing
102     torch itself. Best-effort by design: if torch lives only in a detector subprocess
env
103     (native WASB), this returns ``False`` in the main process ? which is why the startup
checks
104     treat a GPU mismatch as a WARNING (the runner healthcheck is the authority), never a
hard
105     error that could false-fail."""

```

```

106         return _cuda_available()
107
108
109     def suggest_profile(
110         env: Optional[Mapping[str, str]] = None, *, probe_cuda: bool = False
111     ) -> str:
112         """SUGGEST (never auto-apply ? D15) a deployment profile from the environment.
113
114         Signals, in priority order:
115
116         - ``K_SERVICE`` set (Cloud Run)           ? ``"cloud-serving"``
117         - ``CI`` truthy                             ? ``"cpu-mock"``
118         - ``probe_cuda`` and a CUDA GPU present ? ``"truly-local"``
119         - otherwise                                ? ``""`` (no strong suggestion)
120
121         ``probe_cuda`` defaults to ``False`` so the common config-load path stays cheap and
122         never
123         imports torch; a CLI ``--detect`` or cloud-serving startup that wants the GPU probe
124         passes
125         ``probe_cuda=True``. The result is advisory only: the caller decides whether to act
126         on it.
127
128         """
129         e = os.environ if env is None else env
130         if e.get("K_SERVICE"):
131             return "cloud-serving"
132         if _truthy(e.get("CI")):
133             return "cpu-mock"
134         if probe_cuda and _cuda_available():
135             return "truly-local"
136         return ""

```

## 20. user (2026-06-21T13:35:13.126Z)

```

311: class StorageConfig(BaseModel):
312:     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a locally
313:     MOUNTED
314:     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to the
315:     constructor ?
316:     output_root: str = ""
317:     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
318:     `backend`/`output_root`).
319:     # input name is resolved against ``input_root`` (e.g.
320:     ``input_root``="gs://bucket/videos"`` + ".mp4")
321:     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
322:     ``input_backend``
323:     input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
324:     input_root: str = ""
325:     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
326: class DeploymentConfig(BaseModel):
327:     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
328:     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
329:     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)

```

## 21. assistant (2026-06-21T13:35:16.047Z)

Let me read the StorageConfig and DeploymentConfig models in full to capture every field, then check how the worker and API boundary use these.

## 22. user (2026-06-21T13:35:17.489Z)

```

305         cf_access_aud: str = "" # the CF Access application AUD tag the token must
306         carry
307         # CORS allow-origins. Default ["*"] keeps the local web UI working; tighten to the

```

```

hosted
307 # origin(s) in multi-tenant mode. (credentials stay OFF ? Cf-Access is a header, not
a cookie.)
308 cors_allow_origins: List[str] = ["*"]
309
310
311 class StorageConfig(BaseModel):
312     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314     + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315     Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316     **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317     (boto3/google auth chains supply credentials). See ``backend/storage/``.
318     backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319     # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320     output_root: str = ""
321     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322     # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324     # input name is resolved against ``input_root`` (e.g.
``input_root``="gs://bucket/videos"`` + "x.mp4"
325     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326     # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329     input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
330     input_root: str = ""
331     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
332     # One folder per video keyed by the MD5 video_id:
<root>/[<workspace_prefix>]/<video_id>/...
333     # so a local deploy and a cloud (bucket) deploy share ONE relative layout (only the
root
334     # differs). DEFAULT-OFF: convergence is phased + flipped via a Launch Report (D5).
When OFF
335     # the legacy flat layout is used unchanged.
336     single_folder_per_video: bool = False
337     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
338     workspace_root: str = ""
339     # Cloud namespace prefix under the root so per-video folders sit tidily beside
340     # videos/ labels/ weights/ (e.g. gs://bucket/workspaces/<video_id>/). "" = root
directly.
341     workspace_prefix: str = "workspaces"
342     local: dict[str, Any] = Field(default_factory=dict)
343     s3: dict[str, Any] = Field(default_factory=dict)
344     gcs: dict[str, Any] = Field(default_factory=dict)
345     gdrive: dict[str, Any] = Field(default_factory=dict)
346
347
348 class DeploymentConfig(BaseModel):
349     """Deployment profile ? one switch that selects a coherent bundle of compute +
storage
350     knobs (COMPUTE_DECOUPLED_SERVING M1, ``docs/COMPUTE_DECOUPLED_SERVING/README.md``
§4.3).
351

```

```

352     **Pure / additive:** the empty default profile (``"``) applies NO preset, so the
pipeline
353     behaves exactly as before this section existed. A profile is opt-in (``config.json``
354     ``deployment.profile`` or the ``DEPLOYMENT_PROFILE`` env var); auto-detect only ever
355     *suggests* one (D15 ? cloud spend is never entered silently). Nothing in the core
reads
356     these fields yet (M1): selecting a profile works purely by pre-setting the EXISTING
knobs
357     (``indexing.detector_impl`` / ``skip_ai_handoff``, ``storage.backend``) in the
loader.
358
359     Valid values mirror ``backend/config/presets.py`` (parity-pinned by a unit test)."""
360
361     # "" = no profile (today's manual knobs). Others apply presets.PROFILE_PRESETS.
362     profile: Literal["", "truly-local", "cloud-serving", "cpu-mock"] = ""
363     # Where the core runs. Set by the profile preset; overridable in config.json. Inert
in M1
364     # (recorded for observability; consumed by the ComputeBackend seam from M2+).
365     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
366
367
368     class BillingConfig(BaseModel):
369         """Per-job cost ledger (COMPUTE_DECOUPLED_SERVING M8, D8).
370
371         **DEFAULT-OFF:** ``enabled=False`` ? nothing records cost (the v6 cost columns stay
NULL =
372         today's behaviour). When enabled, a job's cost is computed + stored in **USD** via
the versioned
373         ``pricebook`` DB table and **displayed in INR** at ``fx_inr_per_usd``. The default
pricebook
374         version is the seeded ``placeholder-v0`` (every rate ``0.0`` ? cost ``0``) until the
owner
375         inserts a real, calibrated version and points ``pricebook_version`` at it.
376
377         ? Even with ``enabled=True`` + a real pricebook, recorded cost stays **0** until a
follow-up
378         wires the worker to emit per-job ``usage`` (gpu/wall/egress/gemini) ? M8 lands the
ledger +
379         pricebook + the recording seam; usage emission (the metering hooks) is a separate
step."""
380
381         enabled: bool = False
382         pricebook_version: str = "placeholder-v0" # the seeded $0 version; owner flips to
a real one
383         fx_inr_per_usd: float = 83.0 # USD?INR display rate (Bangalore
market)
384         margin: float = 0.0 # resale markup (?0); 0 = bill at
cost
385
386
387     class AppConfig(BaseModel):
388         """Root configuration object.
389
390         All runtime configuration should be accessed via an instance of this class
391         (or the loader) rather than raw dict lookups.
392         """
393
394         sport: Sport = "badminton"
395         ai_provider: str = "gemini"
396         # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable
flagship;
397         # the alias never deprecates underneath us. Eval reports record the run date
alongside.
398         ai_model: str = "gemini-flash-latest"
399

```



```

400     validation: ValidationConfig = Field(default_factory=ValidationConfig)
401     indexing: IndexingConfig = Field(default_factory=IndexingConfig)
402     output: OutputConfig = Field(default_factory=OutputConfig)
403     stitching: StitchingConfig = Field(default_factory=StitchingConfig)
404     security: SecurityConfig = Field(default_factory=SecurityConfig)
405     logging: LoggingConfig = Field(default_factory=LoggingConfig)
406     storage: StorageConfig = Field(default_factory=StorageConfig)
407     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
408     billing: BillingConfig = Field(default_factory=BillingConfig)
409
410     # Allow extra keys during transition so existing config.json files do not break.
411     model_config = {
412         "extra": "allow",
413         "validate_assignment": True,
414     }

```

### 23. user (2026-06-21T13:35:18.055Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8     ``local://``
9     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10    local runs
11    are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13    key) ?
14    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17    CLI uses
18    (validator registry ? segmenter ? report), never a forked pipeline.
19    """
20    from __future__ import annotations
21
22    import argparse
23    import csv
24    import json
25    import logging
26    import os
27    import sys
28    import tempfile
29    from typing import Any, List
30
31    from backend import storage
32    from backend.compute import ComputeJobSpec, get_compute_backend
33    from backend.config import (
34        StartupCheckError,
35        enforce_startup_checks,
36        load_config,
37        probe_cuda_available,
38    )
39    from backend.infrastructure.database import Database
40    from backend.pipeline.segmenters.base import segmenter_registry
41    from backend.pipeline.validators.base import registry as validator_registry
42    from backend.reporting import emit_indexer_report
43    from backend.results import Failure
44    from backend.utils.hashing import compute_video_id
45
46    logger = logging.getLogger("backend.worker")

```

```

44
45     def _coerce(v: str) -> Any:
46         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
47         str."""
48         low = v.lower()
49         if low in ("true", "false"):
50             return low == "true"
51         for cast in (int, float):
52             try:
53                 return cast(v)
54             except ValueError:
55                 pass
56         return v
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
60         indexing.skip_ai_handoff=true)."""
61         for kv in sets or []:
62             key, sep, val = kv.partition("=")
63             if not sep:
64                 raise ValueError(f"--set expects k=v, got {kv!r}")
65             parts = key.split(".")
66             obj = config
67             for p in parts[:-1]:
68                 obj = getattr(obj, p)
69                 setattr(obj, parts[-1], _coerce(val))
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
73         friendly
74         artifact (same schema as the rally-annotator labels)."""
75         with open(path, "w", newline="") as f:
76             w = csv.writer(f)
77             w.writerow(["start", "end", "shot_count", "ending_reason"])
78             for s in segments:
79                 w.writerow([
80                     f'{float(s.get("start_time", 0.0)):.3f}',
81                     f'{float(s.get("end_time", 0.0)):.3f}',
82                     s.get("shot_count", ""),
83                     s.get("ending_reason", s.get("shot_count_confidence", "")),
84                 ])
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
87     None:
88         with open(path, "w") as f:
89             json.dump({
90                 "video_id": video_id,
91                 "status": status,
92                 "segment_count": len(segments),
93                 "segments": [{"start": float(s.get("start_time", 0.0)),
94                             "end": float(s.get("end_time", 0.0))} for s in segments],
95             }, f, indent=2)
96
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
99         CUDA
100         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
101         aborts
102         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
103         no-op
104         (returns True, ZERO work) when no deployment.profile is selected.

```

```

102
103     The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104     torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105     WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106     no-op of work, not just of output."""
107     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108     cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109     try:
110         enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111         return True
112     except StartupCheckError:
113         return False # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
120
121         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
122         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
123         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
124         from backend.infrastructure.execution_policy import PolicyTimeout
125
126         spec = ComputeJobSpec(
127             video_uri=args.video_uri, out_uri=args.out_uri,
128             segmenter=args.segmenter, config_overrides=tuple(args.set or ()),
129         )
130         try:
131             result = compute.run_infer(spec)
132         except PolicyTimeout as e:
133             logger.warning("[worker] remote compute did not complete (no marker within
budget): %s", e)
134             return 4
135             logger.info("[worker] remote compute done: video_id=%s rc=%d outputs=%s",
136                         result.video_id, result.returncode, result.outputs_uri)
137             return result.returncode
138
139
140     def _write_done_marker(done_uri: str, video_id: str, returncode: int, segment_count:
int,
141                          status: str, storage_cfg: dict, scratch: str) -> None:
142         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
143         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return."""
144         try:
145             marker = {"video_id": video_id, "returncode": returncode,
146                     "segment_count": segment_count, "status": status}
147             local = os.path.join(scratch, "_done.json")
148             with open(local, "w", encoding="utf-8") as f:
149                 json.dump(marker, f)
150             storage.put(local, done_uri, config=storage_cfg)
151             logger.info("[worker] wrote completion marker -> %s", done_uri)
152         except Exception as e: # never fail a good run because the marker push hiccuped

```

```

153         logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
154
155
156     def cmd_infer(args: argparse.Namespace) -> int:
157         config = load_config(args.config) if args.config else load_config()
158         _apply_overrides(config, args.set)
159         if not _run_startup_checks(config):
160             return 2
161
162         # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
163         # process.
164         # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
165         # through to the
166         # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
167         # (the
168         # dispatcher forces compute_target=local_gpu), so it never recursively re-
169         # dispatches.
170         compute = get_compute_backend(config)
171         if compute is not None:
172             return _dispatch_remote(compute, args)
173
174         storage_cfg = config.storage.model_dump()
175
176         scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
177         os.makedirs(scratch, exist_ok=True)
178         config.output.output_dir = scratch
179
180         # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
181         # to scratch.
182         local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"),
183         config=storage_cfg)
184         print(f"[worker] pulled {args.video_uri} -> {local_video}")
185
186         # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
187         # byte-identical).
188         video_id = compute_video_id(local_video)
189
190         # 3. VALIDATE (same registry as the main CLI).
191         val_results, val_context = validator_registry.run_all_with_context(local_video,
192         config)
193         failures = [r for r in val_results if not r.passed]
194         db = Database(os.path.join(scratch, config.output.db_name))
195         db.add_video(video_id, local_video, "failed" if failures else "processed",
196         [r.model_dump() for r in val_results])
197
198         segments: List[dict] = []
199         segmenter_actual = None
200         segmentation_ran = False
201         status = "failed"
202         try:
203             if failures:
204                 print("[worker] validation FAILED: "
205                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
206                 return 2
207             seg_name = args.segmenter or config.indexing.default_segmenter
208             segmenter_cls = segmenter_registry.get(seg_name)
209             if not segmenter_cls:
210                 print(f"[worker] unknown segmenter: {seg_name!r} "
211                 f"(have {list(segmenter_registry.list_available())})")
212                 return 2
213             segmenter_actual = seg_name
214             result = segmenter_cls(db, config).process_video(video_id, local_video,
215             max_frames=args.max_frames)
216             segmentation_ran = True
217             if isinstance(result, Failure):

```

```

209         print(f"[worker] segmenter FAILED: {result.message}")
210         return 3
211     segments = result.value if hasattr(result, "value") else result
212     status = "success"
213     # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
214     # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
215     # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
216     if not segments:
217         detector_impl = (os.environ.get("RALLY_DETECTOR_IMPL")
218                         or getattr(config.indexing, "detector_impl", None) or
"auto")
219         logger.warning(
220             "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
221             "The detector substrate yielded no usable trajectories/windows ? check
the detector "
222             "logs above; common causes: native runner UNAVAILABLE
(weights/repo/WASB_PYTHON env), "
223             "the WASB env produced no detections, or the input resolution differs
from the tuned "
224             "proxy resolution. Re-run with -v for the native command + frame
counts.",
225             video_id, segmenter_actual, detector_impl,
226         )
227     finally:
228         # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
229         emit_indexer_report(
230             db=db, video_id=video_id, video_path=local_video, config=config,
231             val_results=val_results, val_context=val_context,
232             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
233             was_reingest=False, segmentation_ran=segmentation_ran,
234         )
235
236     # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
237     out_local = os.path.join(scratch, "outputs", video_id)
238     os.makedirs(out_local, exist_ok=True)
239     _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
240     _write_report_json(video_id, segments, status, os.path.join(out_local,
"report.json"))
241
242     out_prefix = args.out_uri.rstrip("/")
243     pushed = []
244     for fn in sorted(os.listdir(out_local)):
245         uri = f"{out_prefix}/outputs/{video_id}/{fn}"
246         storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
247         pushed.append(uri)
248
249     print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):")
250     for u in pushed:
251         print("    ", u)
252
253     # M6: a remote dispatcher passes --done-uri; write a completion marker so its
bucket-poll
254     # terminates (carries video_id ? the dispatcher's output path + the worker rc).
DEFAULT-OFF:
255     # no --done-uri ? no marker (unchanged local run). Written on SUCCESS only ? a
failed container
256     # job currently lets the dispatcher's bounded poll time out (a failure-marker is a
follow-up).
257     if getattr(args, "done_uri", None):

```

```

258         _write_done_marker(args.done_uri, video_id, 0, len(segments), status,
storage_cfg, scratch)
259     return 0
260
261
262     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
263     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
264     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
265
266
267     def _probe_video_fps_width(path: str):
268         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
269
270         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
271         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
272         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
273         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
274         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
275         rather than guesses.
276         """
277         import json
278         import subprocess
279         try:
280             out = subprocess.check_output(
281                 ["ffprobe", "-v", "error", "-select_streams", "v:0",
282                  "-show_entries", "stream=r_frame_rate,width", "-of", "json", path],
283                 stderr=subprocess.DEVNULL).decode()
284             st = (json.loads(out).get("streams") or [{}])[0]
285             num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
286             fps = float(num) / float(den or "1")
287             width = float(st.get("width") or 0)
288             if fps > 0 and width > 0:
289                 return fps, width
290             except (subprocess.SubprocessError, ValueError, KeyError, OSError,
json.JSONDecodeError):
291                 pass
292             return None
293
294
295     def _resolve_train_detector(args: argparse.Namespace, config: Any):
296         """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
297         detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
298         stub=CPU mock). Returns the runner, or prints an error + returns None.
299
300         Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the
301         detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
302         no shuttle detector and is rejected.
303         """
304         from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
305
306         seg_cls = segmenter_registry.get(args.segmenter)
307         if not seg_cls:
308             print(f"[worker] unknown segmenter: {args.segmenter!r} "
309                   f"(have {list(segmenter_registry.list_available())})")
310             return None

```

```

311     seg = seg_cls(None, config)
312     if not isinstance(seg, TrajectoryHybridSegmenter):
313         print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
314               f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / "
fusion_hybrid).")
315         return None
316     try:
317         runner, _ = seg._resolve_runner(config.get("indexing", {}))
318     except NotImplementedError as e:
319         # e.g. detector_impl='native' for a family without a native runner (tracknet, or
320         # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
traceback.
321         print(f"[worker] detector not available: {e}")
322         return None
323     ok, msg = runner.healthcheck()
324     if not ok:
325         print(f"[worker] detector environment not ready: {msg}")
326         return None
327     print(f"[worker] detector ready ({args.segmenter}): {msg}")
328     return runner
329
330
331 def cmd_train(args: argparse.Namespace) -> int:
332     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
shell out
333     to training.gen0.harness (backend must NOT import training) -> push the artifact
bundle.
334
335     Two trajectory sources (the train pipeline + harness are identical either way):
336     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
synthetic
337         shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
CI.
338     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
339     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
This
340         is the M3.5 "first real training" path; only the trajectory source flips.
341     """
342     import subprocess
343
344     config = load_config(args.config) if args.config else load_config()
345     _apply_overrides(config, args.set)
346     if not _run_startup_checks(config):
347         return 2
348     storage_cfg = config.storage.model_dump()
349
350     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
351     labels_dir = os.path.join(scratch, "labels")
352     traj_dir = os.path.join(scratch, "trajectories")
353     videos_dir = os.path.join(scratch, "videos")
354     os.makedirs(labels_dir, exist_ok=True)
355     os.makedirs(traj_dir, exist_ok=True)
356
357     corpus = args.corpus_uri.rstrip("/")
358     label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/",
config=storage_cfg)
359                        if u.lower().endswith(".csv"))
360     if len(label_uris) < 2:
361         print(f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
362               f"under {corpus}/labels/")
363         return 2
364
365     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.

```

```

366     runner = None
367     video_by_stem: dict = {}
368     if args.trajectory_source == "detector":
369         os.makedirs(videos_dir, exist_ok=True)
370         runner = _resolve_train_detector(args, config)
371         if runner is None:
372             return 2
373         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
374             vname = storage.parse_uri(vuri).name
375             if not vname.lower().endswith(_VIDEO_EXTS):
376                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
377             video_by_stem[os.path.splitext(vname)[0]] = vuri
378
379     print(f"[worker] trajectory source: {args.trajectory_source}")
380     specs: List[dict] = []
381     for uri in label_uris:
382         fname = storage.parse_uri(uri).name # e.g.
GX020094_proxy.rallies.csv
383         name = fname.replace(".rallies.csv", "").replace(".csv", "")
384         local_label = storage.fetch(uri, os.path.join(labels_dir, fname),
config=storage_cfg)
385         if args.trajectory_source == "detector":
386             vuri = video_by_stem.get(name)
387             if not vuri:
388                 print(f"[worker] no video for {name!r} under {corpus}/videos/ ?
skipping.")
389                 continue
390             local_video = storage.fetch(vuri, os.path.join(videos_dir,
storage.parse_uri(vuri).name),
391                                     config=storage_cfg)
392             video_id = compute_video_id(local_video)
393             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
394             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
395             meta = _probe_video_fps_width(local_video)
396             if meta is None:
397                 print(f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess).")
398                 continue
399             fps, frame_width = meta
400             traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
401             if not traj:
402                 print(f"[worker] detector produced no trajectory for {name!r} ?
skipping.")
403                 continue
404             specs.append({"name": name, "trajectory": traj, "labels": local_label,
405                          "fps": fps, "frame_width": frame_width})
406         else:
407             from backend.pipeline.detectors.stub_runner import
synth_trajectory_from_labels
408             traj = os.path.join(traj_dir, f"{name}_traj.csv")
409             synth_trajectory_from_labels(local_label, traj, fps=args.fps,
frame_width=args.frame_width)
410             specs.append({"name": name, "trajectory": traj, "labels": local_label,
411                          "fps": args.fps, "frame_width": args.frame_width})
412
413     if len(specs) < 2:
414         print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}.")
415         return 2
416     manifest_path = os.path.join(scratch, "manifest.json")
417     with open(manifest_path, "w", encoding="utf-8") as f:
418         json.dump(specs, f, indent=2)

```



```

419     src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock
stub"
420     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
421
422     out_dir = os.path.join(scratch, "artifacts")
423     cmd = [sys.executable, "-m", "training.gen0.harness",
424            "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
425     if args.floor:
426         floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"),
config=storage_cfg)
427                        if "://" in args.floor else args.floor)
428         cmd += ["--floor", floor_local]
429     print("[worker] training:", " ".join(cmd))
430     rc = subprocess.run(cmd).returncode
431     if rc != 0:
432         print(f"[worker] gen0 harness failed (rc={rc})")
433         return rc
434
435     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
436     bundle = os.path.join(out_dir, args.version)
437     out_prefix = args.out_uri.rstrip("/")
438     pushed = []
439     for fn in sorted(os.listdir(bundle)):
440         uri = f"{out_prefix}/weights/{args.version}/{fn}"
441         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
442         pushed.append(uri)
443     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
444     for u in pushed:
445         print("    ", u)
446     return 0
447
448
449 def build_parser() -> argparse.ArgumentParser:
450     p = argparse.ArgumentParser(prog="python -m backend.worker",
451                                description="Storage-aware worker: pull ? run pipeline ?
push.")
452     p.add_argument("-v", "--verbose", action="store_true",
453                   help="DEBUG logging (the native detector command, frame counts, per-
stage detail)")
454     sub = p.add_subparsers(dest="cmd", required=True)
455
456     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
457     pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// /
gcs://")
458     pi.add_argument("--out-uri", required=True, help="output prefix
(outputs/<video_id>/? is appended)")
459     pi.add_argument("--segmenter", default=None, help="default:
indexing.default_segmenter")
460     pi.add_argument("--config", default=None, help="config.json path (default:
./config.json)")
461     pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp
dir)")
462     pi.add_argument("--max-frames", type=int, default=None)
463     pi.add_argument("--done-uri", default=None,
464                   help="M6: storage URI for a completion MARKER (video_id + rc)
written on a "
465                   "successful run, so a remote (Cloud Run) dispatcher's bucket-
poll terminates. "
466                   "Default-OFF: omit it for the normal local run.")
467     pi.add_argument("--set", action="append", default=[], metavar="k=v",
468                   help="config override, e.g. --set indexing.skip_ai_handoff=true")
469     pi.set_defaults(func=cmd_infer)
470
471     pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] ->
trajectories -> harness)")

```

```

472     pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+
videos/ for "
473         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
474     pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/
is appended)")
475     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
476     pt.add_argument("--trajectory-source", choices=["synth", "detector"],
default="synth",
477         help="synth = CPU-mock label-consistent trajectories (default, no
GPU); "
478         "detector = REAL shuttle trajectories via the resolved
DetectorRunner "
479         "(needs videos/ in the corpus; honours detector_impl).")
480     pt.add_argument("--segmenter", default="wasb_hybrid",
481         help="trajectory hybrid whose detector is used for --trajectory-
source detector")
482     pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or
storage URI)")
483     pt.add_argument("--config", default=None)
484     pt.add_argument("--scratch", default=None)
485     pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps
(detector probes the video)")
486     pt.add_argument("--frame-width", type=float, default=1280.0,
487         help="synth trajectory frame width (detector probes the video)")
488     pt.add_argument("--set", action="append", default=[], metavar="k=v")
489     pt.set_defaults(func=cmd_train)
490     return p
491
492
493     def _setup_logging(verbose: bool) -> None:
494         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
diagnostics are
495         VISIBLE ? the worker previously ran with no logging config, so every
segmenter/detector log
496         (detector choice, 'Running native WASB inference', availability failures) was
silently dropped.
497         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
(the CLI
498         entry), not at import, so importing the module never reconfigures a caller's
logging."""
499         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
library
500         # configured logging at import), which would leave the level at WARNING and re-
suppress the
501         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
verbosity.
502         logging.basicConfig(
503             level=logging.DEBUG if verbose else logging.INFO,
504             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
505             datefmt="%H:%M:%S",
506             force=True,
507         )
508
509
510     def main(argv=None) -> int:
511         args = build_parser().parse_args(argv)
512         _setup_logging(getattr(args, "verbose", False))
513         return args.func(args)
514
515
516     if __name__ == "__main__":
517         sys.exit(main())
518

```

Now let me check how the API boundary (the FastAPI server / `/api/process`) uses input resolution, and how the loader supplies config from env. Let me also confirm which cloud SDKs are mentioned in pyproject extras.

## 25. user (2026-06-21T13:35:24.261Z)

```
174:     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
194:         db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump() for r
in val_results])
199:             "results": [r.model_dump() for r in val_results],
205:         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for r
in val_results])
216:             "results": [r.model_dump() for r in val_results],
245:             "results": [r.model_dump() for r in val_results],
258:             "results": [r.model_dump() for r in val_results],
297:     input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
664:         input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
671:     local_video = storage.acquire_local_input(args.video_path, storage_cfg)
683:     db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])
```

## 26. user (2026-06-21T13:35:25.703Z)

```
35:     "google-genai>=2.4.0",
74:# install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare R2 +
77:storage-s3 = ["boto3>=1.34"]
78:storage-gcs = ["google-cloud-storage>=2.16"]
```

## 27. user (2026-06-21T13:35:27.128Z)

Found 2 files  
backend\api\uploads.py  
backend\api\static\app.js

## 28. assistant (2026-06-21T13:35:29.651Z)

Let me look at the API process handler around the input resolution and the pyproject extras section, plus how google-cloud-run is declared (it's not in extras).

## 29. user (2026-06-21T13:35:30.602Z)

```
160         the sync endpoint used to return (UI compatibility); the per-video observability
161         report is still emitted in `finally:` and grafted as response["reports"].
162
163         `progress` is an optional callable(stage: str) used to surface coarse job progress.
164         Raises on unexpected internal errors ? the caller records those as a job failure
165         (after this function has already restored the pre-run segments, see below).
166         """
167         notify = progress or (lambda stage: None)
168
169         notify("hashing")
170         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
171         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
172         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
173         # consumers (hash / validators / segmenter) use the resolved local path.
174         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
175         video_id = compute_video_id(local_video)
176         was_reingest = db_instance.get_video(video_id) is not None
```

```

177
178     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
179     notify("validating")
180     logger.info(f"Running validations for {req.video_path}...")
181     val_results, val_context = registry.run_all_with_context(local_video, global_config)
182     failures = [r for r in val_results if not r.passed]
183
184     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
185     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
186     seg_name = req.segmenter or default_seg
187     segmenter_actual = None
188     segmentation_ran = False
189     response: Optional[Dict[str, Any]] = None
190     try:
191         if failures:
192             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
193             # status; it no longer destroys the video's prior good segments.
194             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
195             response = {
196                 "video_id": video_id,
197                 "status": "failed",
198                 "message": "Video failed validation checks.",
199                 "results": [r.model_dump() for r in val_results],
200             }
201             return response
202
203     # 2. Run segmenter & indexer
204     logger.info("[API] Video passed checks. Segmenting and indexing...")
205     db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
206
207     segmenter_cls = segmenter_registry.get(seg_name)
208     if not segmenter_cls:
209         # Submit-time validation already 400s unknown names; this is the defensive
210         # in-worker path (e.g. config default pointing at an unregistered
segmenter).
211         available = list(segmenter_registry.list_available().keys())
212         response = {
213             "video_id": video_id,
214             "status": "failed",
215             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
216             "results": [r.model_dump() for r in val_results],
217             "play_segments": [],
218         }
219         return response

```

### 30. user (2026-06-21T13:35:31.570Z)

```

285     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
missing or
286     # spoofed identity header fails here with 401, before any work) and tags the job's
owner_id.
287     try:
288         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
289     except edge_auth.EdgeAuthError as e:
290         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e
291
292     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
293     try:
294         validate_input_path(req.video_path, allowed_root=allowed_root)

```

```

295         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
296         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
297         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
298         except ValueError as e:
299             raise HTTPException(status_code=400, detail=str(e)) from e
300         # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
301         # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
302         # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
303         # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
304         # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
305         # rejects a non-local URI as out-of-root.
306         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
307             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
308         if req.segmenter and not segmenter_registry.get(req.segmenter):
309             available = list(segmenter_registry.list_available().keys())
310             raise HTTPException(
311                 status_code=400,
312                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
313         )
314

```

### 31. user (2026-06-21T13:35:32.384Z)

```

60         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
61     ]
62
63     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
64     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
65     # express, so they are NOT declarable here. Install them out-of-band instead:
66     # # CUDA 12.4 box:
67     # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
68     # # CPU-only box (also what CI uses):
69     # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
70     gpu = []
71
72     # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
73     # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
74     # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
75     # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
76     # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
77     storage-s3 = ["boto3>=1.34"]
78     storage-gcs = ["google-cloud-storage>=2.16"]
79
80     [project.scripts]
81     # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
82     indexer = "backend.main:main"
83
84     [tool.hatch.metadata]
85     # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
86     # reference*); hatchling requires opting in to allow those in metadata.
87     allow-direct-references = true
88
89     [tool.hatch.build.targets.wheel]

```

```
90     # `backend` is a namespace package (no __init__.py); list both import roots
91     # explicitly so an editable/wheel install exposes `import backend` and
92     # `import training` regardless of cwd.
93     packages = ["backend", "training"]
94
```

### 32. assistant (2026-06-21T13:35:35.251Z)

I now have a complete picture. Let me verify one important gap: `google-cloud-run` (the real CloudRun client SDK) is NOT declared in any extra. Let me confirm.

### 33. user (2026-06-21T13:35:37.321Z)

No matches found

### 34. user (2026-06-21T13:35:38.094Z)

```
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47         "PyJWT[crypto]>=2.8", # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
48     ]
49
50     # M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when
security.edge_auth_enabled=True.
51     # `jwt` is lazy-imported, so the base runtime works without this; install it for
hosted/multi-tenant.
52     auth = ["PyJWT[crypto]>=2.8"]
53
54     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
55     # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
56     # fetched over SSH; if you lack access, simply omit this group ? reporting
57     # tests skip via pytest.importorskip. For local dev against a sibling checkout:
58     #   pip install -e ../sports-obsreport --no-build-isolation
59     reporting = [
60         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
61     ]
```

### 35. assistant (2026-06-21T13:35:41.360Z)

I've confirmed `google-cloud-run` is not declared in any extra (it's installed out-of-band on the control plane, as noted in `cloud\_run.py:69-71`). I now have everything needed to answer the questions comprehensively.

### 36. user (2026-06-21T13:36:47.891Z)

Structured output provided successfully

### 37. assistant (2026-06-21T13:36:51.997Z)

I've completed the investigation of the storage and compute backend registries and returned my findings via StructuredOutput. The analysis covers backend selection (URI-scheme for storage, `compute\_target` for compute), credential supply (env/ADC chains, never config), the CloudRunCompute dispatch+bucket-poll flow, lazy SDK imports, and the packaging implications for a downloadable app.